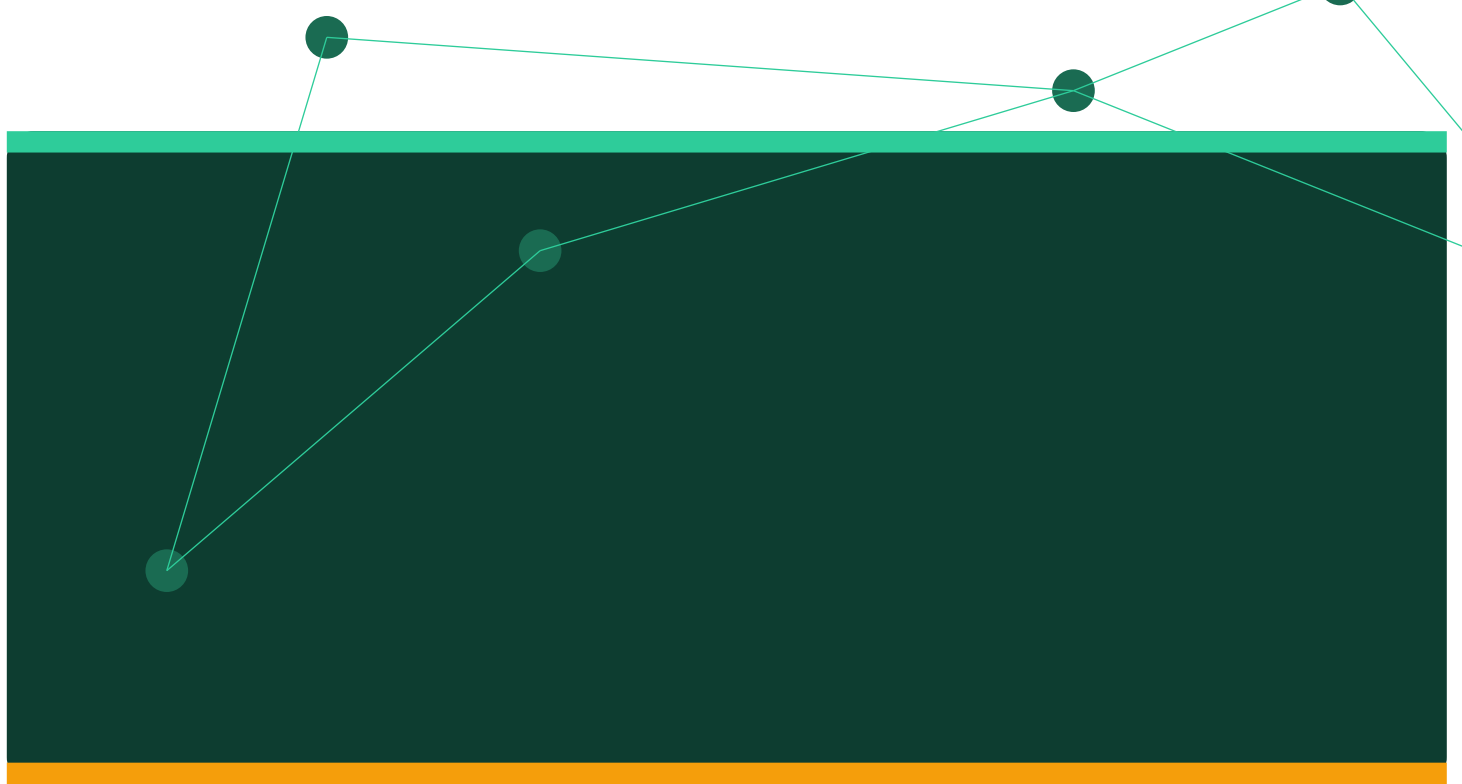




Week 2 — OpenAI Agents SDK

A Deep Dive into Asynchronous Programming, Multi-Agent Frameworks & Practical Applications

What You Will Learn

AsyncIO & Coroutines • OpenAI Agents SDK Core Concepts • Building Your First Agent • Tools vs. Handoffs • Guardrails & Safety • Multi-Model Integration • SDR Agent Project • Deep Research Agent

SECTION 1 — THEORY

1. What is Agentic AI?

Traditional AI systems operate in a simple **input** → **output** pattern: you give them a prompt, they generate a response, and the interaction ends. **Agentic AI** fundamentally changes this model. An AI agent is a system that can **perceive its environment, reason about goals, take a sequence of actions, and adapt its behaviour based on feedback** — all without requiring a human to direct every step.

Think of the difference this way: a traditional LLM is like a brilliant consultant you can call once for advice. An AI Agent is like hiring a full employee — you give them a goal, and they figure out the subtasks, use the right tools, coordinate with teammates, and report back when done.

Traditional LLM vs. Agentic AI

Dimension	Traditional LLM	Agentic AI
Interaction Model	Single turn: prompt → response	Multi-turn: plan → act → observe → adapt
Decision Making	Stateless; no memory between calls	Stateful; tracks goals, context, history
Tool Usage	None by default	Can call APIs, search web, run code, send emails
Collaboration	Single model	Multiple specialised agents working together
Human Oversight	Required for every step	Minimal; guardrails handle safety automatically
Use Case	Q&A;, summarisation, generation	Research, sales automation, complex workflows

2. Asynchronous Python & AsyncIO

Before you can build effective multi-agent systems, you need to understand the concurrency model that powers them: **AsyncIO**. This is the single most important prerequisite concept for working with the OpenAI Agents SDK.

2.1 The Problem: Why Concurrency Matters

Imagine you are building an SDR (Sales Development Representative) agent that needs to simultaneously search LinkedIn for leads, query a CRM, draft personalised emails, and wait for responses from three different LLM APIs. If your program runs these tasks **sequentially** (one after another), you are wasting enormous amounts of time just sitting idle while waiting for network responses. A typical LLM API call might take 2–10 seconds — if you have 10 concurrent operations, sequential execution costs you 20–100 seconds, while concurrent execution collapses that to roughly the duration of the single longest task.

■ **IMPORTANT**

Concurrency ≠ Parallelism. Parallelism uses multiple CPU cores simultaneously (multiprocessing). Concurrency uses a single thread but intelligently switches between tasks during idle periods (I/O waits). AsyncIO is concurrency, not parallelism.

2.2 Three Core Pillars of AsyncIO

1

Event Loop

The runtime orchestrator. It maintains a queue of all pending coroutines and tasks, decides which one runs next, and switches control between

2

async def

Defines a coroutine — a special kind of function that can pause its own execution mid-way and hand control back to the event loop. Calling an async function

3

await

The pause-and-hand-off operator. When a coroutine hits an await statement, it pauses itself and tells the event loop: 'I am waiting for this to

2.3 Synchronous vs. Asynchronous: A Visual Comparison

Consider a program that makes three API calls, each taking 3 seconds:

Synchronous (Sequential)

call_api_1() → waits 3s call_api_2() →
waits 3s call_api_3() → waits 3s Total: 9
seconds

Asynchronous (Concurrent)

await call_api_1() ■■ await call_api_2() ■■
all run concurrently await call_api_3() ■■
Total: ~3 seconds

■ TIP

In multi-agent pipelines where multiple LLM calls happen in parallel — each potentially taking several seconds — AsyncIO can reduce total latency by 3x to 10x compared to sequential execution.

3. OpenAI Agents SDK — Core Architecture

The **OpenAI Agents SDK** is an open-source Python framework specifically designed for building production-grade agentic AI applications. Its design philosophy is **lightweight, flexible, and out-of-the-way** — it handles the complex orchestration plumbing so you can focus on agent logic and business goals.

Importantly, despite being published by OpenAI, the SDK is **not limited to OpenAI models**. It can integrate with Gemini, Groq, Anthropic, and any other provider that exposes a compatible API interface. This makes it a truly model-agnostic orchestration layer.

3.1 The Three Pillars of the SDK

Agents

An Agent is the fundamental unit of the SDK. It is an AI entity with a defined identity (name), behaviour (instructions/system prompt), capability (model), and optionally: tools it can use, other agents it can hand off to, output formats it should produce, and guardrails that constrain its behaviour. Agents are declarative — you describe what the agent should be, and the SDK handles execution.

Handoffs	A Handoff is a mechanism for transferring control from one agent to another. When Agent A performs a handoff to Agent B, execution responsibility shifts completely to Agent B. This enables specialisation: a triage agent can route to a billing specialist, a research coordinator, or a creative writer depending on the task. Control flow can branch, chain, or loop across agents.
Guardrails	Guardrails are safety and correctness checks applied at the boundaries of agent execution — either at the input (before the agent processes a request) or at the output (after the final agent produces a result). They can validate formats, detect harmful content, enforce business rules, or trigger fallback behaviours. Note: In the current SDK version, guardrails only work with OpenAI models.

3.2 The Three Steps to Run an Agent

Every agent execution in the SDK follows the same three-step lifecycle:

Step 1	Step 2	Step 3
Create an Agent	Configure the Runner	Execute & Consume
Define name, instructions (system prompt), model, tools, and handoffs.	The Runner orchestrates the agent loop: call LLM, handle tool calls, process handoffs, repeat.	await Runner.run() for full output, or stream events progressively for real-time UX.

4. Advanced Agentic Workflows: Tools vs. Handoffs

As your agent systems grow more complex, one of the most important architectural decisions you will face is: **should this capability be a Tool or a Handoff?** Both enable agents to extend their reach, but they differ fundamentally in how they handle control flow.

Tools A Tool is a Python function decorated with <code>@tool</code> that the agent can choose to invoke to interact with the external world — search the web, query a database, send an email, run Python code, call a REST API, etc. Mechanically: the LLM generates a JSON object describing the function call (name + arguments). The SDK executes the actual Python function and returns the result back to the LLM. The LLM then incorporates that result into its next response. Crucially, control returns to the original agent after every tool call. Use Tools when: you need to interact with external systems but the main orchestrating agent should stay in control. ■ <i>Best for: API calls, web search, email sending, database queries</i> Example: Agent uses SendGrid tool to send an email, then continues its workflow.	Handoffs A Handoff is a mechanism for an agent to transfer responsibility to another agent. Unlike a tool call, control does NOT automatically return to the original agent. The new agent takes over and continues the conversation from where the previous one left off. Handoffs are used for building specialised pipelines: a Triage Agent that routes requests, a Researcher that gathers data and hands off to a Writer, or a multi-language support system where the routing agent hands off to language-specific agents. Use Handoffs when: you want to delegate an entire responsibility to a specialist, especially in sequential or branching workflows. ■ <i>Best for: specialist routing, sequential pipelines, role-based delegation</i> Example: Orchestrator hands off to a Billing Specialist agent for complex refunds.
--	--

5. Multi-Model Integration & Guardrails

5.1 Multi-Model Architecture

One of the most powerful architectural patterns in the Agents SDK is using **different models for different agents in the same pipeline**. This is called multi-model integration, and it allows you to optimise for cost, speed, and capability simultaneously.

Agent Role	Suggested Model	Why
Triage / Router	GPT-4o-mini or Gemini Flash	Fast, cheap — just needs to classify intent
Deep Researcher	GPT-4o or Claude Opus	Complex reasoning, long context, nuanced analysis
Email Drafter	GPT-4o-mini	Creative writing, cost-effective at scale
Code Generator	GPT-4o or DeepSeek	Strong code synthesis and debugging capabilities
Validator / Guardrail	Any OpenAI model	Guardrails only work with OpenAI models (SDK constraint)

5.2 Guardrails — Safety at Scale

Guardrails are the SDK's built-in mechanism for enforcing safety, correctness, and business-rule compliance. They are essentially validation checks that intercept and inspect data at two points in the agent lifecycle:

Type	Where Applied	Purpose	Example
Input Guardrail	Before agent processes user input	Block harmful, off-topic, or malformed requests	Reject inputs containing PII or prompt injections
Output Guardrail	After final agent produces output	Validate format, tone, or content of agent's response	Ensure email draft is in JSON with required fields

■ CAUTION

Current SDK constraint: Guardrails only work with OpenAI models. If your pipeline uses Gemini or Groq agents, place an OpenAI model agent at the input/output boundary specifically to serve as the guardrail layer.

6. Structured / Desired Output

By default, agents return free-form text. In production pipelines, however, you almost always need the output to conform to a specific schema so the next agent or application can consume it reliably. The SDK lets you define a **Pydantic model** as the desired output type for any agent. The SDK then instructs the underlying LLM to produce output that matches that schema — and automatically validates and parses it for you.

This is particularly powerful in multi-agent pipelines where Agent A's structured output directly feeds into Agent B's input — enabling clean, type-safe data flow between agents without brittle string parsing.

7. Real-World Projects Overview

7.1 Project: SDR (Sales Development Representative) Agent

This project demonstrates a sophisticated, production-style agentic system for automating the initial stages of a B2B sales process. The system combines three tiers of agentic architecture to show how real enterprise pipelines are built.

Layer	Component	Responsibility
Layer 1	Workflow Design	Map out the overall agent graph: which agents exist, how they connect, what data flows between them, and what the terminal output looks like.
Layer 2	Agent Using a Tool	The email-drafting agent invokes a SendGrid tool to send transactional emails directly from within the agent pipeline without human intervention.
Layer 3	Agent Calling Other Agents	The orchestrator agent hands off to specialist sub-agents: one for lead research, one for personalisation, one for email composition — each with their own model, instructions, and tools.

7.2 Project: Deep Research Agent

The Deep Research Agent is a demonstration of agentic AI at its most powerful: an autonomous agent capable of conducting internet research, synthesising findings from multiple sources, and generating comprehensive, structured reports — all without human guidance beyond the initial research query.

Capability	Implementation Detail
Internet Search	Agent uses a web search tool (e.g., Tavily, SerpAPI) to retrieve live information
Structured Parsing	Search results are parsed and fed back as structured context for the LLM
Multi-step Reasoning	Agent iteratively refines its research: search → read → identify gaps → search again
Output Generation	Final report produced in structured format: sections, citations, key findings

SECTION 2 — PRACTICAL & CODING

The following notebook cells cover all key SDK concepts with executable code. Each cell includes a purpose statement, a line-by-line breakdown, expected output, and a key insight to solidify your understanding.

Cell [1]

AsyncIO — Coroutines & the Event Loop

Purpose

Demonstrate the fundamental `async/await` pattern: defining coroutines, running them on the event loop, and visualising how concurrent execution saves time.

Cell [1] — AsyncIO Fundamentals

```
import asyncio
import time

# --- Synchronous version (for comparison) ---
def sync_fetch(name, delay):
    print(f' [SYNC] Starting {name}...')
    time.sleep(delay) # blocks the entire program
    print(f' [SYNC] Done {name}')
```



```
# --- Asynchronous coroutine ---
async def async_fetch(name, delay):
    print(f' [ASYNC] Starting {name}...')
    await asyncio.sleep(delay) # yields control; non-blocking
    print(f' [ASYNC] Done {name}')
```



```
# --- Run coroutines concurrently with asyncio.gather ---
async def main():
    start = time.time()
    await asyncio.gather(
        async_fetch('LLM_API_Call_1', 3),
        async_fetch('LLM_API_Call_2', 2),
        async_fetch('LLM_API_Call_3', 4),
    )
    elapsed = time.time() - start
    print(f'Total time: {elapsed:.2f}s (expected ~4s, not ~9s)')
```



```
# Entry point for async programs
```

```
asyncio.run(main())
```

■ OUTPUT / CONSOLE

```
[ASYNC] Starting LLM_API_Call_1...
[ASYNC] Starting LLM_API_Call_2...
[ASYNC] Starting LLM_API_Call_3...
[ASYNC] Done      LLM_API_Call_2
[ASYNC] Done      LLM_API_Call_1
[ASYNC] Done      LLM_API_Call_3
Total time: 4.01s (expected ~4s, not ~9s)
```

Line-by-Line Breakdown

import asyncio — The standard library module that provides the event loop, the `gather()` function, coroutine support, and all async primitives. No pip install required — it ships with Python 3.5+.

async def async_fetch(name, delay): — The `async` keyword before `def` marks this function as a coroutine. Calling `async_fetch()` does NOT run the function body; it returns a coroutine object. Only when the event loop schedules and runs it does execution begin.

await asyncio.sleep(delay) — This is the key difference from `time.sleep()`. `asyncio.sleep()` is a non-blocking sleep: when a coroutine awaits it, that coroutine suspends itself and the event loop is free to run any other ready coroutine. `time.sleep()` would freeze the entire program.

asyncio.gather(...) — Schedules all three coroutines to run concurrently on the same event loop. They all 'start' immediately and interleave during their sleep periods. The `gather` completes when the longest-running coroutine finishes.

asyncio.run(main()) — The standard entry point for async programs. It creates a fresh event loop, runs the given coroutine to completion, and then closes the loop. In Jupyter notebooks, you use `'await main()'` directly instead (no `asyncio.run` needed).

■ **Key Insight:** Notice the output order — all three coroutines **START** before any finish. The event loop switches between them during their sleep periods. **Call_2 (2s) finishes first, then Call_1 (3s), then Call_3 (4s).** Total wall time is only 4s — the maximum, not the sum. This 3x speedup is exactly what makes AsyncIO essential for multi-agent AI.

Cell [2]

Building Your First Agent — The Jokerster

Purpose

Create and run a minimal but complete agent using the OpenAI Agents SDK — understanding every parameter and how the runner lifecycle works.

Cell [2] — Your First Agent

```
# Install the SDK first (run in terminal, not notebook cell)
# pip install openai-agents python-dotenv

import asyncio
from dotenv import load_dotenv
```



```

from agents import Agent, Runner

# Load OPENAI_API_KEY from a .env file
load_dotenv(override=True)

# Step 1: Define the Agent
jokester = Agent(
    name='Jokester',
    instructions='You are a professional comedy writer. Tell short, witty jokes.',
    model='gpt-4o-mini', # cost-effective for simple tasks
)

# Step 2 + 3: Runner orchestrates execution
async def main():
    result = await Runner.run(
        agent=jokester,
        input='Tell me a joke about Python programming.',
    )
    print('Agent Output:')
    print(result.final_output)

# Run the async main function
asyncio.run(main())

# In Jupyter: just use await main() directly

```

■ OUTPUT / CONSOLE

```

Agent Output:
Why do Python developers prefer dark mode?
Because light attracts bugs! ■

```

Line-by-Line Breakdown

from agents import Agent, Runner — Imports the two core SDK classes. Agent is the definition object (what the agent is). Runner is the execution engine (how the agent runs). They are intentionally separated so you can define agents declaratively and reuse them across multiple runner invocations.

Agent(name=..., instructions=..., model=...) — The three minimum required parameters. 'name' is a human-readable identifier for logging and handoff routing. 'instructions' is the system prompt — this is where you define the agent's persona, task, and constraints. 'model' specifies which LLM backbone to use.

instructions vs. input — A critical distinction: 'instructions' is the persistent system prompt (the agent's identity and role). 'input' (passed to Runner.run) is the specific task or user message for this particular execution. Think of instructions as the job description, and input as the current task.

await Runner.run(...) — This is asynchronous! The Runner sends the system prompt + input to the LLM, processes the response, handles any tool calls or handoffs in a loop, and returns a RunResult object when the

agent signals it is done. `result.final_output` contains the agent's final text response.

■ **Key Insight:** The Agent object is declarative and reusable — you create it once and can run it thousands of times with different inputs. The `model` parameter accepts any model string, including 'gemini-1.5-pro' or 'groq/llama3-70b' — making the SDK truly model-agnostic beyond just OpenAI.

Purpose

Demonstrate how to define Python functions as tools that agents can invoke, enabling them to interact with the external world beyond pure text generation.

Cell [3] — Agents with Tools

```
import asyncio

from agents import Agent, Runner, tool
from dotenv import load_dotenv

load_dotenv(override=True)

# --- Define Tools as Python functions with @tool decorator ---

@tool
def search_company_info(company_name: str) -> str:
    '''Search for information about a company for sales outreach.'''
    # In production: call SerpAPI, Tavily, or LinkedIn API here
    return f'Company: {company_name} | Industry: SaaS | Size: 200 employees'

@tool
def send_email(to: str, subject: str, body: str) -> str:
    '''Send an outreach email via SendGrid.'''
    # In production: call SendGrid API here
    print(f' [EMAIL SENT] To: {to} | Subject: {subject}')
    return 'Email delivered successfully.'

# --- Agent with tools ---

sdr_agent = Agent(
    name='SDR_Agent',
    instructions=(
        'You are a sales development representative. '
        'Research the company, then draft and send a personalised outreach email.'
    ),
    model='gpt-4o-mini',
    tools=[search_company_info, send_email], # tools list
)

async def main():
    result = await Runner.run(
```

```
agent=sdr_agent,  
input='Reach out to TechCorp Inc. about our AI automation platform.',  
)  
print('Final Output:', result.final_output)  
  
asyncio.run(main())
```

■ OUTPUT / CONSOLE

```
[EMAIL SENT] To: contact@techcorp.com | Subject: AI Automation for TechCorp  
Final Output: I've researched TechCorp Inc. (a 200-person SaaS company) and  
sent a personalised outreach email highlighting how our AI platform can  
help them automate their sales workflows. Email delivered successfully.
```

Line-by-Line Breakdown

@tool decorator — This is the SDK's mechanism for converting a regular Python function into an agent-callable tool. Under the hood, the SDK inspects the function's signature and docstring to automatically generate a JSON Schema that the LLM uses to understand when and how to call this tool. The docstring is especially important — it IS the tool's description seen by the LLM.

Type annotations (company_name: str) -> str — These are not optional cosmetic choices. The SDK reads these annotations to build the JSON Schema. Proper type hints ensure the LLM generates valid arguments for your tool. Use Pydantic models for complex structured inputs.

tools=[search_company_info, send_email] — The agent receives both tools and the LLM decides autonomously which tool to call, when to call it, and in what order — based solely on the task in its instructions and input. You do not hardcode the call sequence; the LLM reasons about it.

Tool return values — Whatever a tool function returns (as a string or JSON-serialisable object) is fed back to the LLM as the 'tool result'. The LLM then uses that information to decide on the next action. Control always returns to the same agent after each tool call.

■ **Key Insight: The docstring is your tool's API contract with the LLM. Write it like documentation — be explicit about what the tool does, what parameters it expects, and what it returns. Poor docstrings = poor tool usage by the LLM.**

Purpose

Build a multi-agent system using handoffs where an orchestrator agent routes requests to specialist agents, demonstrating agent-to-agent delegation.

Cell [4] — Multi-Agent Handoffs

```
import asyncio

from agents import Agent, Runner, handoff
from dotenv import load_dotenv

load_dotenv(override=True)

# --- Specialist Agents ---
research_agent = Agent(
    name='Research_Specialist',
    instructions='You are a market researcher. Provide detailed company analysis.',
    model='gpt-4o', # more capable model for complex reasoning
)

email_agent = Agent(
    name='Email_Specialist',
    instructions='You write concise, compelling B2B sales emails. Max 150 words.',
    model='gpt-4o-mini', # cheaper model for creative writing
)

# --- Orchestrator with handoffs ---
orchestrator = Agent(
    name='SDR_Orchestrator',
    instructions=(
        'You coordinate the sales process. '
        'First hand off to Research_Specialist for company intel, '
        'then hand off to Email_Specialist to draft the outreach.'
    ),
    model='gpt-4o-mini',
    handoffs=[
        handoff(research_agent),
        handoff(email_agent),
    ],
)
```

```

async def main():
    result = await Runner.run(
        agent=orchestrator,
        input='Prepare an outreach campaign for DataViz Corp.',
    )
    print(result.final_output)

asyncio.run(main())

```

■ OUTPUT / CONSOLE

[Orchestrator → Research_Specialist]

DataViz Corp Analysis: Series B startup, 150 employees, data visualisation SaaS, currently expanding into enterprise segment. Pain point: manual reporting workflows.

[Research_Specialist → Email_Specialist]

Subject: Eliminate Manual Reporting at DataViz Corp

Hi [Name], I saw DataViz Corp is scaling into enterprise – congrats!

Our AI automation platform has helped similar SaaS companies cut reporting time by 70%. Worth a 15-min call? – [Your Name]

Line-by-Line Breakdown

Two different models for two agents — `research_agent` uses `gpt-4o` for complex analytical reasoning; `email_agent` uses `gpt-4o-mini` for cost-effective creative writing. This is multi-model integration in practice: right model for right task. Total cost is reduced without sacrificing quality where it matters.

handoff(research_agent) — This registers `research_agent` as a possible handoff target for the orchestrator. The SDK generates a special `'transfer_to_research_specialist'` tool that the orchestrator's LLM can call when it decides to delegate. The agent name in the handoff must match the receiving agent's name exactly.

handoffs=[...] — The orchestrator's handoff list. The LLM decides which handoff to trigger based on its instructions and the current conversation context. This is where the orchestrator's instructions become critical — they guide when and to whom it should hand off.

Control flow — Unlike tool calls, handoffs transfer execution responsibility. Orchestrator runs → hands off to Research → Research runs → hands off to Email → Email produces final output. The runner tracks this entire chain and returns the last agent's output as `result.final_output`.

■ **Key Insight: The handoff pattern is the foundation of production-grade agentic systems. Real-world examples: customer support triage → billing agent → refund agent; research coordinator → data agent → analyst → writer → reviewer. Each specialist agent can be independently improved, tested, and swapped without touching the others.**

Purpose

Implement an input guardrail that intercepts and blocks inappropriate or off-topic requests before they reach the main agent, demonstrating how to build safe production-ready pipelines.

Cell [5] — Input Guardrails

```
import asyncio

from agents import Agent, Runner, input_guardrail, GuardrailFunctionOutput
from pydantic import BaseModel
from dotenv import load_dotenv

load_dotenv(override=True)

# --- Define the guardrail check output schema ---
class SalesRelevanceCheck(BaseModel):
    is_sales_related: bool
    reasoning: str

# --- The guardrail agent (always OpenAI model) ---
guardrail_agent = Agent(
    name='Input_Validator',
    instructions=(
        'Check if the user input is related to B2B sales or business outreach. '
        'Return JSON with is_sales_related (bool) and reasoning (str).'
    ),
    output_type=SalesRelevanceCheck,
    model='gpt-4o-mini', # must be OpenAI model for guardrails
)

# --- The guardrail function ---
@input_guardrail
async def sales_topic_guardrail(ctx, agent, input_text):
    result = await Runner.run(guardrail_agent, input_text)
    check = result.final_output
    return GuardrailFunctionOutput(
        output_info=check,
        tripwire_triggered=not check.is_sales_related,
    )
```

```
# --- Main agent with guardrail ---

sdr_agent = Agent(
    name='SDR_Agent',
    instructions='You are a B2B sales assistant.',
    model='gpt-4o-mini',
    input_guardrails=[sales_topic_guardrail],
)

async def main():
    # Test 1: Valid request
    result = await Runner.run(sdr_agent, 'Reach out to TechCorp about our product.')
    print('Test 1:', result.final_output)

    # Test 2: Off-topic (should be blocked)
    try:
        result = await Runner.run(sdr_agent, 'Write me a poem about the ocean.')
    except Exception as e:
        print(f'Test 2 BLOCKED: Guardrail triggered - {type(e).__name__}')

asyncio.run(main())
```

■ OUTPUT / CONSOLE

```
Test 1: I'll reach out to TechCorp about our product right away...
Test 2 BLOCKED: Guardrail triggered - InputGuardrailTripwireTriggered
```

Line-by-Line Breakdown

class SalesRelevanceCheck(BaseModel) — A Pydantic model defining the structured output schema for our guardrail agent. By setting `output_type` on the `guardrail_agent`, the SDK forces the LLM to respond in this exact format, enabling reliable programmatic access to `is_sales_related` as a boolean.

@input_guardrail decorator — Marks this async function as an input guardrail. The SDK calls it automatically before the main agent processes any input. It receives three parameters: `ctx` (run context), `agent` (the guarded agent), and `input_text`.

tripwire_triggered=not check.is_sales_related — The tripwire is the guardrail's verdict. When `True`, the SDK immediately raises an `InputGuardrailTripwireTriggered` exception, preventing the main agent from executing at all. No LLM cost is incurred for the blocked request beyond the guardrail check itself.

input_guardrails=[sales_topic_guardrail] — Attaches the guardrail to the agent. Multiple guardrails can be chained — they run in order, and if any trips, execution stops immediately. Output guardrails use `output_guardrails=[...]` and run after the agent produces its response.

■ **Key Insight: Guardrails are a second AI layer watching your main AI layer. This 'AI watching AI' pattern is how enterprise-grade safety is implemented. In production, you might stack multiple guardrails: one for safety (no harmful content), one for topic relevance, one for PII detection — all running before the main agent sees the input.**

Purpose

Configure an agent to return a structured Pydantic object instead of raw text, enabling reliable downstream consumption in multi-agent pipelines.

Cell [6] — Structured Output

```
import asyncio

from agents import Agent, Runner
from pydantic import BaseModel
from typing import List
from dotenv import load_dotenv

load_dotenv(override=True)

# --- Define the desired output schema ---
class LeadProfile(BaseModel):
    company_name: str
    industry: str
    estimated_size: str
    pain_points: List[str]
    outreach_angle: str
    recommended_subject_line: str

# --- Agent configured with output_type ---
research_agent = Agent(
    name='Lead_Researcher',
    instructions=(
        'You research B2B leads. For the given company, return a structured '
        'lead profile with industry, size, pain points, and outreach angle.'
    ),
    model='gpt-4o',
    output_type=LeadProfile, # enforce structured output
)

async def main():
    result = await Runner.run(
        agent=research_agent,
        input='Research DataStream Analytics for SDR outreach.',
    )
```

```

profile = result.final_output # already a LeadProfile object!

print(f'Company: {profile.company_name}')
print(f'Industry: {profile.industry}')
print(f'Pain Points:')
for p in profile.pain_points:
    print(f' - {p}')
print(f'Subject: {profile.recommended_subject_line}')

asyncio.run(main())

```

■ OUTPUT / CONSOLE

```

Company:  DataStream Analytics
Industry: Data & Analytics SaaS
Pain Points:
  - Manual data pipeline maintenance consuming engineering bandwidth
  - Difficulty scaling real-time dashboards to enterprise clients
  - High cloud costs due to unoptimised query patterns
Subject:  Cut Your Data Pipeline Costs by 40% – Worth 15 Minutes?

```

Line-by-Line Breakdown

class LeadProfile(BaseModel) — A Pydantic data class defining the exact structure the agent must return. Each field has a type annotation. The SDK converts this into a JSON Schema and instructs the LLM (via the API's structured outputs feature) to produce a response that matches it exactly.

pain_points: List[str] — Demonstrates that nested types are supported. The LLM will produce a JSON array of strings for this field. More complex types like nested BaseModels, Optional fields, and Enum values are all supported by Pydantic and the SDK.

output_type=LeadProfile — The SDK parameter that activates structured output. Internally, the SDK uses OpenAI's JSON mode / response_format API feature to enforce the schema at the model level, not just via prompting.

profile = result.final_output — Because we set output_type, the SDK automatically parses and validates the LLM's JSON response and returns a fully instantiated LeadProfile Python object. You access fields like profile.company_name with full IDE autocomplete and type safety.

■ **Key Insight: Structured outputs are the glue of multi-agent pipelines. Agent A's LeadProfile output becomes the direct input for Agent B's email drafting. No brittle string parsing, no regex, no JSON.loads() with try/except everywhere — just clean Python objects flowing between agents. This is how you build reliable production systems.**

Purpose

Integrate all concepts — tools, handoffs, structured output, and multi-model integration — into a complete, end-to-end SDR pipeline that demonstrates production-level agentic design.

Cell [7a] — SDR Pipeline: Setup, Schema & Tools

```
import asyncio

from agents import Agent, Runner, tool, handoff
from pydantic import BaseModel
from dotenv import load_dotenv

load_dotenv(override=True)

# --- Structured Output Schema ---
class EmailDraft(BaseModel):
    to: str
    subject: str
    body: str
    call_to_action: str

# --- Tools ---
@tool
def search_lead(company: str) -> str:
    '''Search for company info and decision maker contact.'''
    return f'{company}: Series A SaaS, CEO=John Smith'

@tool
def send_email(to: str, subject: str, body: str) -> str:
    '''Send outreach email via SendGrid API.'''
    print(f' >> EMAIL SENT to {to}')
    return 'Sent'
```

Cell [7b] — SDR Pipeline: Agents, Orchestrator & Execution

```
# --- Specialist Agents (different models per role) ---

researcher = Agent(
    name='Researcher', model='gpt-4o',
    instructions='Research the company and find the decision maker.',
    tools=[search_lead],
)

writer = Agent(
```

```

name='Email_Writer', model='gpt-4o-mini',
instructions='Draft a personalised, concise B2B outreach email.',
output_type=EmailDraft,
)

sender = Agent(
name='Sender', model='gpt-4o-mini',
instructions='Send the drafted email using the send_email tool.',
tools=[send_email],
)

# --- Orchestrator ties it all together ---

orchestrator = Agent(
name='SDR_Orchestrator', model='gpt-4o-mini',
instructions=(
'Run the SDR pipeline: 1) Handoff to Researcher, '
'2) Handoff to Email_Writer, 3) Handoff to Sender.'
),
handoffs=[handoff(researcher), handoff(writer), handoff(sender)],
)

async def main():
result = await Runner.run(
agent=orchestrator,
input='Run full SDR pipeline for CloudMetrics Inc.',
)
print('Pipeline complete:', result.final_output)

asyncio.run(main())

```

■ OUTPUT / CONSOLE

```

>> EMAIL SENT to john@cloudmetrics.com
Pipeline complete: SDR pipeline executed successfully for CloudMetrics Inc.
Research complete → Email drafted → Email delivered to john@cloudmetrics.com.
Subject: Scale CloudMetrics Without Scaling Your Team

```

Line-by-Line Breakdown

Three distinct specialist agents with different models — Researcher uses gpt-4o (complex reasoning for analysis), writer and sender use gpt-4o-mini (cost-effective for creative and simple tasks). This is the optimal multi-model strategy: invest compute where it matters, save cost where it doesn't.

writer with output_type=EmailDraft — The Email_Writer agent produces a structured EmailDraft object. The orchestrator and sender agent can rely on this guaranteed structure — no parsing failures, no missing fields.

Orchestrator with handoffs=[researcher, writer, sender] — The orchestrator acts as a traffic controller. Its instructions define the pipeline sequence. The LLM follows those instructions and triggers handoffs in order. If you need a different order for a different campaign type, you just change the orchestrator's instructions — no code changes needed.

The full pipeline autonomously — Once `Runner.run()` is called with the orchestrator and the initial input, the entire chain runs without any further human intervention: `research` → `draft` → `send`. This is agentic automation in action.

■ **Key Insight: This pattern — Orchestrator + Specialist Agents + Tools + Structured Output — is the architectural blueprint for virtually every production agentic system. Replace 'SDR' with 'Customer Support', 'Code Review', 'Report Generation', or 'Research Summarisation' and the same pattern applies. Master this pattern and you can build any agentic application.**

CHAPTER SUMMARY

Concept	Key Takeaway
Agentic AI	AI that can plan, act, use tools, and collaborate — not just respond
AsyncIO	Non-blocking concurrency via event loop, <code>async def</code> , and <code>await</code> — cuts multi-API latency 3-10x
Agents	Declarative AI entities: name + instructions + model + tools + handoffs
Runner	Execution orchestrator: handles the LLM loop, tool calls, and handoff routing
Tools	Python functions the LLM can call; control returns to calling agent after each call
Handoffs	Transfer execution responsibility to specialist agents; enables pipeline architectures
Guardrails	Input/output safety checks using a secondary AI validator; only on OpenAI models
Structured Output	<code>Pydantic output_type</code> enforces schema compliance — the glue of multi-agent pipelines
Multi-Model	Different agents can use different models — optimise cost vs. capability per task
SDR Project	Orchestrator + Specialist + Tools pattern — the universal blueprint for agentic apps